

AI-Powered Legal Document Analysis System: Clause Detection, Risk Classification and Plain-English Summarization on CUAD

by

Jerin Aktar
22101279

Afnan Mazumdar
24141229

Shoyeb Hasan Sayem
22101386

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
July 2026

Declaration

It is hereby declared that

1. The thesis submitted is our own original work while completing our degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Jerin Aktar

22101279

Afnan Mazumdar

24141229

Shoyeb Hasan Sayem

22101386

Approval

The thesis titled “AI-Powered Legal Document Analysis System: Clause Detection, Risk Classification and Plain-English Summarization on CUAD” submitted by

1. Jerin Aktar (22101279)
2. Afnan Mazumdar (24141229)
3. Shoyeb Hasan Sayem (22101386)

of Summer 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering on July 2026.

Examining Committee:

Supervisor:

(Member)

Utsho Kumar Roy
Department of Computer Science and Engineering
Brac University

Head of Department:

(Chair)

Sadia Hamid Kazi, PhD
Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Ethics Statement

This work uses the Contract Understanding Atticus Dataset (CUAD), a publicly released research dataset of 510 commercial contracts taken from public filings and annotated by legal experts under The Atticus Project. We did not collect or process any private, confidential, or personally sensitive documents, and no human subjects were involved. All experiments ran on our own hardware. The system we built is a research prototype meant to assist contract review; it is not legal advice, and we state this limitation in the application itself. The third-party models we fine-tuned (DistilBERT, FLAN-T5) and all supporting libraries are used under their open-source licenses, and every outside source is cited.

Abstract

Commercial contracts often run past a hundred pages, and missing a single clause can cost a signer dearly. This thesis builds a three-model pipeline for automated contract review on CUAD, a dataset of 510 expert-annotated contracts covering 41 clause categories. A presence classifier first decides which categories a contract contains: we combine a full-document TF-IDF baseline with a sliding-window DistilBERT trained under multiple-instance learning, and an ensemble that requires both to agree reaches 85.5% accuracy and 0.776 micro-F1 on a held-out 20% contract-level test split. A DistilBERT question-answering model then pulls out the exact clause text (token-F1 0.763), and a fine-tuned FLAN-T5 rewrites each clause in plain English (ROUGE-L 0.775). Every detected clause also gets a High/Medium/Low risk label scored from the weaker party’s side. The pipeline is deployed in *Clausify*, a web application that turns raw contract text into a risk-prioritized report.

Keywords: legal NLP; CUAD; clause extraction; transformer models; risk classification; contract summarization

Acknowledgement

First of all, all praise to the Almighty for allowing us to complete this thesis without any major interruption. We are grateful to our supervisor, Utsho Kumar Roy, for his guidance and patience through every phase of this work, and especially for insisting on a strict held-out evaluation — that one requirement shaped how honestly this thesis reports its results. Finally, we thank our families for their constant support and encouragement. Without them, this work would not have been possible.

Contents

Declaration	ii
Approval	iii
Ethics Statement	iv
Abstract	v
Acknowledgement	vi
Table of Contents	ix
List of Tables	x
List of Figures	xiii
List of Acronyms	xiv
Glossary	xv
1 Introduction	1
1.1 Background and Motivation	1
1.2 System at a Glance	1
1.3 Problem Statement	2
1.4 Research Objectives	2
1.5 Contributions	3
1.6 Thesis Organization	3
2 Literature Review	5
2.1 Legal NLP and the CUAD Benchmark	5
2.2 Transformer Encoders and Model Distillation	5

2.3	Long-Document Strategies and Multiple-Instance Learning	6
2.4	Text-to-Text Models for Summarization	7
2.5	Gap Addressed by This Thesis	7
3	Dataset and Exploratory Analysis	9
3.1	Data Source	9
3.2	Preprocessing: the Wide-to-Long Melt	9
3.3	Exploratory Findings	10
3.4	The 80/20 Train/Test Split	10
4	Methodology	12
4.1	Pipeline Overview	12
4.2	Stage 2A: Presence Classification	12
4.2.1	A strong baseline first: full-document TF-IDF	12
4.2.2	Measuring the retrieval ceiling — and dropping retrieval	13
4.2.3	Sliding-window max-pooling (multiple-instance learning)	14
4.2.4	The conjunctive ensemble	15
4.3	Stage 2B: Span Extraction	16
4.4	Stage 1: Plain-Language Summarization	17
4.5	Risk Taxonomy	17
4.6	Leakage Discipline	18
4.7	End-to-End Data Flow	18
5	Model Training	20
5.1	Optimization Schedule	20
5.2	Presence Classifier	21
5.3	Span Extractor	21
5.4	Summarizer	22
5.5	Engineering Notes: Training on Consumer Hardware	22
6	Testing and Results	24

6.1	Presence Classification	24
6.1.1	Model comparison	24
6.1.2	How much of that ranking is real?	24
6.1.3	Confusion-matrix analysis	26
6.1.4	Per-category behaviour	27
6.2	Span Extraction	27
6.3	Summarization	28
6.4	Summary Scorecard	29
6.5	Discussion: the Three Findings	29
7	Clausify: the Demonstration Application	31
7.1	Purpose and Design	31
7.2	User Workflow	31
7.3	How the Models Run Inside the App	31
7.4	Implementation Notes	33
7.5	Honest Limitations of the Demo	33
8	Conclusion and Future Work	35
8.1	Conclusion	35
8.2	Limitations	35
8.3	Future Work	36
	References	37
A	Full Risk Taxonomy	38
B	Repository and Artifact Layout	40

List of Tables

2.1	This thesis positioned against prior CUAD work.	8
3.1	The contract-level 80/20 train/test split.	11
4.1	Retrieval hit-rate on positive (contract, category) pairs — the recall ceiling a retrieval-based design would impose.	14
5.1	Training configurations for the three models.	20
6.1	Presence models on the 20% test set (micro-F1 over all 4,182 test cells). . .	24
6.2	Cluster bootstrap over the 102 test contracts (10,000 resamples). The paired difference is what settles each comparison; $P(> 0)$ is the fraction of resam- ples in which the first model wins.	25
6.3	Confusion matrix of the AND-ensemble on the 20% test set (4,182 decisions). .	26
6.4	Test metrics derived from the confusion matrix.	26
6.5	Span-extraction results on the 2,667 answer-bearing test windows.	28
6.6	Fine-tuned vs. zero-shot summarizer on the same test clause (abridged). . .	29
6.7	Final held-out test results (408 training / 102 test contracts).	29
A.1	The 41 CUAD clause categories with risk level and reason.	38

List of Figures

1.1	The system in one picture. A contract enters as raw text; the pipeline decides which of the 41 CUAD categories are present, extracts the exact clause text for each, and then — the layer this thesis contributes — attaches a risk level and a plain-English rewrite. Figure 4.1 expands this into the models that implement each stage, and Figure 4.8 adds the data shapes that move between them.	2
3.1	Per-category positive rate in the training and test halves of the contract-level split, sorted by training frequency. The two distributions overlap closely across all 41 categories, which supports the balance claim in Table 3.1 far more directly than the aggregate rates alone.	11
4.1	The end-to-end pipeline. Shaded (orange) stages — risk classification, plain-language summarization, and the prioritized report — are the layers this work adds beyond CUAD’s original clause-extraction task.	12
4.2	Why the retrieval ceiling settled the design. <i>Top</i> : the query scores every window and only the top- k go forward; whenever the gold-bearing window is not among them, the pair is lost no matter how good the downstream model is. <i>Bottom</i> : the resulting 64.0% ceiling sits <i>below</i> what the cheap full-document baseline already achieves (0.775, dashed line), so retrieval could not have won — which is why it was dropped before any training time was spent on it.	13
4.3	Sliding-window generation, drawn as a staircase so the geometry is explicit. Each 2,000-character window starts 1,500 characters after the previous one, so consecutive windows always share a 500-character overlap (orange column). Because that overlap is larger than the median annotated clause (196 characters), no clause can fall through the gap between two windows: here the clause lands inside <i>both</i> w_3 and w_4 in full. This is what removes the recall ceiling of Figure 4.2, and it is bought at the cost of scoring every window. A median-length contract produces 22 windows; the mean across the corpus is 35, and the longest contract yields 226.	14

4.4	The multiple-instance learning decision rule. Every window of the contract is scored independently against the same category question by one shared DistilBERT, and the document-level probability is the maximum over windows. Here only the shaded window w_2 actually contains the clause, and its score alone determines the document-level answer. One firing window is enough to call the category present — which is what makes recall complete, and equally what makes precision fragile, since a single spurious window anywhere in a long contract produces a document-level false positive. . . .	15
4.5	The conjunctive ensemble. The two presence models see the contract in fundamentally different ways — lexical evidence over the entire document versus semantic evidence over overlapping windows — and their errors are therefore largely uncorrelated. Requiring agreement (AND) discards the false positives that max-pooling introduces. The micro-F1 values on the right are the held-out results reported in Chapter 6, shown here only to indicate which rule the analysis selects.	16
4.6	The two window sizes, drawn to the same scale. Stage 2A tiles the document with 2,000-character <i>presence</i> windows to find out <i>whether</i> a clause is there. Stage 2B then trains on a narrower 1,200-character <i>focus</i> window re-centred on the answer, so the span is guaranteed to survive tokenizer truncation. The two are not alternatives and never compete: the first locates a relevant region, the second reads inside it.	17
4.7	Distribution of the 41 CUAD clause categories across the three risk levels. .	18
4.8	The pipeline of Figure 4.1 annotated with the object that crosses each stage boundary, for one median-length contract. The narrowing from <code>float[n, 41]</code> window scores to a <code>bool[41]</code> presence vector is where max-pooling and the AND rule act; everything downstream operates only on the roughly 13 categories that survive, which is what keeps inference fast enough for an interactive application.	19
5.1	Learning-rate schedule for each of the three training runs: linear decay to zero with no warmup phase, over two epochs. The dashed line marks the start of the second epoch. Note the different vertical scales — the summarizer runs at 3×10^{-4} , an order of magnitude above the two DistilBERT runs, which is standard for T5-family models.	21
5.2	Training and validation loss of the window-level presence classifier.	22
5.3	Fine-tuning loss of the FLAN-T5-small summarizer.	22

6.1	micro-F1 of all presence models on the held-out test set.	25
6.2	Confusion-matrix heatmap of the AND-ensemble on the 20% test set. . . .	27
6.3	Per-category F1 on the test set for the twelve strongest categories.	28
7.1	The Clausify input screen. The toggle selects between pasting contract text and uploading a file; the slider sets the presence-confidence threshold. . . .	32
7.2	Results view on the synthetic test contract: the risk summary bar reports 7 High, 15 Medium, and 8 Low risk clauses (30 of the 41 categories detected), and the risk-prioritized report begins with the High-risk group.	33
7.3	High-risk clause cards. Each card combines the risk badge and reason, Model 1’s presence confidence bar, and the exact clause text extracted by Model 2 from the user’s contract — here <i>Uncapped Liability</i> , <i>Non-Compete</i> , <i>Minimum Commitment</i> , and <i>Irrevocable or Perpetual License</i>	34

List of Acronyms

CUAD	Contract Understanding Atticus Dataset
NLP	Natural Language Processing
TF-IDF	Term Frequency–Inverse Document Frequency
LR	Logistic Regression
MIL	Multiple-Instance Learning
BERT	Bidirectional Encoder Representations from Transformers
QA	Question Answering
EM	Exact Match
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
MPS	Metal Performance Shaders (Apple GPU backend)
TP / TN	True Positive / True Negative
FP / FN	False Positive / False Negative
SQuAD	Stanford Question Answering Dataset
IP	Intellectual Property

Glossary

- Clause category** One of the 41 types of contractual provision annotated in CUAD (e.g., *Governing Law*, *Non-Compete*, *Cap on Liability*).
- Contract-level split** A train/test partition that assigns whole contracts to one side, so no clause from a training contract can appear in the test set.
- Presence classification** The task of deciding, for each clause category, whether a contract contains at least one clause of that category.
- Span extraction** The task of locating the exact character span of a clause inside the contract text, formulated as extractive question answering.
- Max-pooling (over windows)** Scoring every sliding window of a document and taking the maximum window score as the document-level prediction.
- Retrieval ceiling** The upper bound that a retrieval step imposes on a downstream model’s recall: clauses in windows the retriever never selects can never be found.
- Weaker party** The contract signer with less bargaining power, from whose perspective the risk taxonomy in this work is scored.

Chapter 1

Introduction

1.1 Background and Motivation

Almost everything people and businesses do together runs on contracts, but actually reading them is a luxury. A typical agreement in our dataset has a median length of about 33,000 characters — roughly 5,000 words — and the longest one crosses 338,000 characters. Somewhere inside all that text sit the provisions that really matter: who owns newly created intellectual property, whether liability has a ceiling, whether the contract quietly renews itself, whether one party is banned from competing after it ends. Getting a lawyer to review such a document takes hours and costs money that individuals and small businesses often do not have. So people sign contracts they have not fully understood, and carry risks they never saw.

The Contract Understanding Atticus Dataset (CUAD) [1] was built to help automate this kind of review. It contains 510 real commercial contracts in which legal experts highlighted every clause belonging to 41 categories that matter in corporate transactions. The annotation effort alone is said to be worth around two million US dollars of attorney time. But CUAD frames the problem purely as extraction: given a category, find the clause text. Once it finds a clause labelled “Uncapped Liability,” it stops. There is no warning about what that clause means for the signer, no plain-language explanation, and no check of how clauses interact.

Our project tries to close that gap. On top of clause detection and extraction, we add a risk taxonomy that scores every category High, Medium, or Low from the weaker party’s point of view, and a summarization model that rewrites extracted clauses as single plain-English sentences. We packaged the whole pipeline into *Clausify*, a web application where a user pastes or uploads a contract and gets back a risk-prioritized clause report within seconds.

1.2 System at a Glance

Before the detailed treatment in Chapter 4, Figure 1.1 gives the shape of the whole system in one picture. The system answers three questions about a contract in order — *what is in it*, *where is it*, and *what does it mean* — and only the first two are what CUAD was built to benchmark. The third stage, shaded, is what this thesis adds.

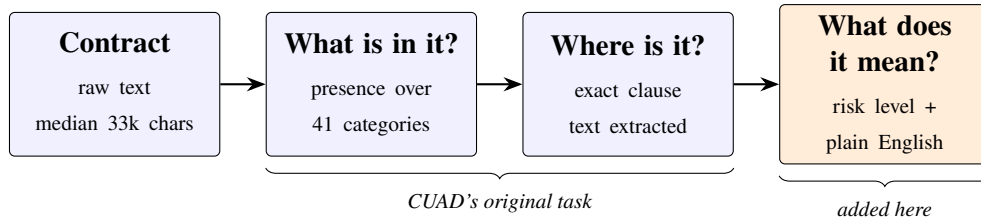


Figure 1.1: The system in one picture. A contract enters as raw text; the pipeline decides which of the 41 CUAD categories are present, extracts the exact clause text for each, and then — the layer this thesis contributes — attaches a risk level and a plain-English rewrite. Figure 4.1 expands this into the models that implement each stage, and Figure 4.8 adds the data shapes that move between them.

1.3 Problem Statement

Concretely, the thesis works on four connected problems:

1. **Presence classification.** For each of the 41 CUAD categories, decide whether a given contract contains at least one clause of that category. The main technical obstacle here is length: contracts are roughly 16× longer than the input window of a standard transformer encoder.
2. **Span extraction.** For every category judged present, locate the exact clause text inside the contract. We formulate this as extractive question answering in the style of SQuAD [8].
3. **Plain-language summarization.** Rewrite each extracted clause as one sentence that a non-lawyer can understand.
4. **Risk communication.** Attach a risk level and a one-line reason to every detected clause, so the output reads as an actionable report rather than a list of quotations.

1.4 Research Objectives

- Build and honestly evaluate a full three-model CUAD pipeline on consumer hardware, using an 80/20 contract-level train/test split so that every reported number comes from genuinely unseen contracts.
- Check whether a windowed transformer actually beats a strong classical baseline for document-level presence classification, instead of assuming that it does.
- Measure span-extraction quality with token-level F1 and exact match on the held-out test set.

- Evaluate the summarizer with both ROUGE-L and a qualitative zero-shot comparison, because we found a single automatic metric can mislead.
- Deploy the trained models in a working demonstration application with a risk-prioritized interface.

1.5 Contributions

The main contributions of this work are:

1. **A risk-aware CUAD framework.** Within the scope of this project, the first extension of CUAD clause detection with a complete High/Medium/Low risk taxonomy covering all 41 categories, scored from the weaker party’s perspective (8 High, 22 Medium, 11 Low).
2. **An honest empirical result on long-document presence classification.** A full-document TF-IDF baseline (micro-F1 0.775) turned out to be surprisingly hard to beat. Our windowed DistilBERT trained with multiple-instance learning loses to it on its own (0.694), and a paired cluster bootstrap confirms that gap is real (95% CI [+0.063, +0.099]). An AND-ensemble of the two, which exploits their different error patterns, nominally finishes first (0.776, accuracy 85.5%) — but the same bootstrap shows its margin over the baseline is indistinguishable from zero, so we report the ensemble for the balanced error profile it produces rather than for its rank.
3. **A methodological guardrail: measure the ceiling before training.** We show that question-guided retrieval windowing caps the attainable recall at 64%, and we used that one measurement to reject the whole retrieval approach before spending any training time on it.
4. **A reproducible, laptop-scale pipeline.** All three models train on a single Apple-Silicon laptop. The split, the trained weights, and a standalone evaluation notebook are all saved, so every number in this thesis can be regenerated from disk.
5. **Clausify**, a working web application that runs the trained models live and presents the results grouped by risk.

1.6 Thesis Organization

Chapter 2 reviews the related literature. Chapter 3 describes the dataset, the preprocessing, and the exploratory analysis that shaped our design choices. Chapter 4 presents the methodology for all pipeline stages. Chapter 5 covers model training, and Chapter 6 reports

the held-out test evaluation, including the confusion-matrix analysis. Chapter 7 describes the Clausify application. Chapter 8 concludes and discusses future work. The appendices contain the full 41-category risk taxonomy and the repository layout.

Chapter 2

Literature Review

2.1 Legal NLP and the CUAD Benchmark

Applying NLP to legal text is an old ambition, but for a long time progress was blocked by a simple problem: legal annotation needs lawyers, and lawyers are expensive. CUAD [1], released at the NeurIPS 2021 Datasets and Benchmarks track by The Atticus Project [2], changed this. It provides 510 real commercial contracts, drawn from public SEC filings, in which experienced attorneys highlighted every span belonging to 41 clause categories that matter in corporate transactions — governing law, non-compete, liability caps, IP assignment, and so on. The dataset ships in the SQuAD format [8]: each (contract, category) pair is posed as a question over the contract text, and the annotated clause spans are the answers, complete with character offsets.

The original CUAD paper treats the task as extractive question answering and reports that performance scales strongly with model size; their best results come from DeBERTa models at the extra-large scale, and even those scores are modest, which says something about how hard legal clause extraction actually is. Two properties of CUAD end up dominating any modelling effort on it. The first is severe class imbalance: some categories appear in nearly every contract, while others show up barely a dozen times in the whole corpus. The second is document length. Contracts are far longer than a transformer’s input limit, so every practical system has to answer the same question first: how does a model with a 512-token window read a 50,000-character contract?

2.2 Transformer Encoders and Model Distillation

The transformer architecture [3] replaced recurrence with self-attention, and BERT-style masked-language-model pre-training [4] established the fine-tuning recipe this thesis depends on: take a model pre-trained on large general corpora, then adapt it to a downstream task with relatively little labelled data. This transfer-learning property matters a lot here, because 510 contracts is a small dataset by deep-learning standards.

Since our compute budget is a single laptop, we use DistilBERT [5], a distilled six-layer version of BERT that keeps roughly 97% of BERT’s language-understanding ability while being 40% smaller and 60% faster. The published CUAD baselines use much larger models,

so our backbone choice trades some headline accuracy for the ability to reproduce everything on ordinary hardware. We point out this scale difference wherever we interpret our results.

2.3 Long-Document Strategies and Multiple-Instance Learning

The usual remedies for the length mismatch come in three families: (i) *retrieval*, where a cheap scorer picks the few windows most likely to hold the answer and only those go to the transformer; (ii) *sliding windows with aggregation*, where the model scores every window and the document-level prediction aggregates the window scores; and (iii) *sparse-attention architectures* such as Longformer, which widen the input window directly.

The third family deserves more than the one-line dismissal it usually gets, because “too expensive” is not on its own a reasoned argument. We rejected sparse attention for three specific reasons, and only the third is about cost. **First, and decisively, it would not actually solve the problem.** Longformer extends the input window to 4,096 tokens. At roughly four characters per token, our median contract of 33,143 characters is about 8,300 tokens and the longest is about 85,000 — so a Longformer would still see only half of a typical contract and a twentieth of the largest one. Sparse attention makes the window eight times wider; it does not make it wide enough. We would still need windowing and an aggregation rule on top, which is precisely the design we ended up with anyway — only with a backbone several times more expensive underneath it. The length problem in CUAD is an order of magnitude past what widening attention fixes. **Second, availability.** Our whole pipeline is built around distilled backbones so that it trains and serves on one laptop, and there is no distilled Longformer with the reliability and tooling support that DistilBERT has; adopting sparse attention would mean giving up the distillation that makes the rest of the project reproducible. **Third, practical cost, of two kinds.** Fine-tuning a 149M-parameter Longformer at 4,096 tokens is far beyond a single Apple-Silicon laptop’s memory budget — and we had already been burned here, since DeBERTa-v3 produced NaN gradients on the MPS backend (Section 5.5), which made us wary of any architecture relying on custom attention kernels on a non-CUDA device. There is also an inference-side cost: *Clausify* scores every window of an uploaded contract live, and the interface is only usable if that finishes in seconds, which rules out a backbone an order of magnitude slower per window. We note that on CUDA hardware with a larger memory budget the first objection would still stand, so this is a design judgement rather than a hardware excuse.

Sliding windows aggregated with a maximum correspond to the classical *multiple-instance learning* (MIL) setting: the document is a bag of window instances, the bag is positive if any instance is positive, and max-pooling implements exactly that rule. MIL has a well-known weakness that our experiments ended up confirming: false-positive inflation. With dozens of windows per document, the chance that *some* window fires by accident grows with

document length, and precision suffers.

2.4 Text-to-Text Models for Summarization

T5 [6] reframes every NLP task as text-to-text, which fits clause simplification naturally: legal text goes in, a plain sentence comes out. FLAN-T5 [7] instruction-tunes T5 on a large mixture of tasks, so it gives usable zero-shot behaviour from a prompt as simple as “summarize in plain English:”. We evaluate summaries with ROUGE-L [9], which measures the longest common subsequence against a reference. Chapter 6 shows why this metric has to be paired with actually reading the outputs when the reference set is small.

2.5 Gap Addressed by This Thesis

Existing CUAD work stops at locating clauses. What a signer actually needs goes one step further: which clauses matter, how much, and what do they mean in plain language. This thesis adds that layer — a per-category risk taxonomy, plain-English summaries, and a deployed application — while also re-examining, on a strict held-out evaluation, the common assumption that a transformer must beat classical lexical methods at the document-level presence decision.

Table 2.1 places this work beside the two reference points above along the dimensions that separate them. Two differences are worth reading off it directly. The obvious one is scope: prior work ends at the span, whereas the output here is a risk-ranked, plain-English report, which is the part a non-lawyer can actually act on. The less obvious one is in the long-document column. Every system on CUAD has to make a choice about document length, but that choice is normally asserted; here it was measured before any training happened — the retrieval ceiling of 64.0% in Section 4.2.2 is what ruled retrieval out and sent us to multiple-instance learning. Note also what the backbone column costs us: our distilled models are roughly an order of magnitude smaller than CUAD’s DeBERTa-v3-XL, so our absolute extraction scores are not comparable to theirs, and we do not present them as such.

Table 2.1: This thesis positioned against prior CUAD work.

Work	Task scope	Backbone	Long-document handling	Risk assessment	Plain-language output	Held-out evaluation
Hendrycks et al. (CUAD, 2021) [1]	Span extraction only	DeBERTa-v3-XL	Full-document QA (large model)	No	No	Yes
Standard retrieval-QA pipelines	Span extraction	Varies	Retrieval + QA (ceiling assumed)	No	No	Varies
This thesis	Presence + span + risk + summary	DistilBERT + FLAN-T5-small	MIL sliding window (ceiling <i>measured</i> , not assumed)	Yes (41-category taxonomy)	Yes (fine-tuned)	Yes (contract-level 80/20)

Chapter 3

Dataset and Exploratory Analysis

3.1 Data Source

All experiments use the official Atticus Project release of CUAD, obtained from the Hugging Face hub ([theatticusproject/cuad](https://huggingface.co/theatticusproject/cuad)). One practical discovery is worth recording here: the repository's default `load_dataset()` configuration exposes only the 511 raw contract PDFs, which cannot be trained on. The annotated data actually lives in two files inside the repository, and each one plays a different role in our pipeline:

- `master_clauses.csv` — an 83-column wide table with one row per contract, which we used for exploratory analysis and for building labels;
- `CUAD_v1.json` — the SQuAD-format file with the full contract text and character-level answer offsets, which we used for span-extraction training and full-document context.

3.2 Preprocessing: the Wide-to-Long Melt

The CSV is wide: 41 categories $\times$ (a clause-text column plus an answer column), with a filename column in front. Turning it into a long table of (contract, category) rows forced two decisions that were not obvious at first:

Pair columns by position, not by name. The header `Notice Period To Terminate Renewal- Answer` contains an inconsistent space, which silently breaks any string matching between a category column and its answer column. Pairing by column position (clause text at odd indices, answer at even) survives typos like this.

Define presence from the clause-text list, not the Yes/No answer. Each clause-text cell holds a stringified Python list, either `[]` or `"['clause', ...]"`. We call a category *present* when this list is non-empty. This rule works the same way across the eight entity-style categories (dates, party names) whose answer column holds a value instead of Yes/No, and it hands us the clause spans as a side effect.

The melt produces $510 \times 41 = 20,910$ labelled (contract, category) rows, of which 32.1% are positive.

3.3 Exploratory Findings

Four properties of the data ended up shaping every design decision that followed:

1. **Severe class imbalance.** Per-category positive rates run from *Document Name* at 100% and *Parties* at 99.8% down to *Source Code Escrow* at 2.5% and *Price Restrictions* at 2.9%. The median category is present in roughly 23% of contracts.
2. **The rare tail is nearly few-shot.** *Source Code Escrow* has only 13 positive contracts in the entire corpus. No model learns categories like that reliably, and per-category scores on this tail are mostly noise. We carry this caveat through the results chapter.
3. **Contracts are long.** Median 33,143 characters (about 5,000 words), maximum 338,211 characters — roughly 16× a transformer’s effective input window. This is the central modelling tension of the whole thesis.
4. **Clauses are short.** The median annotated clause is 196 characters (90th percentile 587, 99th percentile 1,330). So a clause fits easily inside a single window; the hard part is finding it inside a very long document.

It is worth being upfront that 510 contracts is the entire dataset. Its annotations reportedly cost about \$2M of attorney time, so it cannot simply be grown. Three consequences follow: transfer learning from pre-trained models is mandatory; overfitting is the main enemy, so all our conclusions rest on held-out numbers; and the train/test boundary has to be drawn at the contract level.

3.4 The 80/20 Train/Test Split

Following the evaluation protocol set by our supervisor, we split the dataset **80/20 at the contract level**: whole contracts go to one side or the other, so no clause from a training contract can leak into the test set. The split is generated with a fixed seed (42) and *reused identically by all three models*, which means “test” always refers to the same 102 unseen contracts. Table 3.1 summarizes the split; the nearly identical positive rates on both sides confirm it is balanced. We exported the two sides as `data/train_80.csv` and `data/test_20.csv` so the split itself is reproducible.

The aggregate positive rates in Table 3.1 are almost identical (32.0% vs. 32.2%), but a single pair of percentages can hide a badly skewed split — two sides can agree overall while disagreeing sharply category by category. Figure 3.1 therefore compares the two sides for all 41 categories at once. The profiles track each other closely across the whole frequency range (Pearson $r = 0.986$, mean absolute difference 3.4 percentage points). The visible

Table 3.1: The contract-level 80/20 train/test split.

	Contracts	(Contract, category) rows	Positive rate
Train (80%)	408	16,728	32.0%
Test (20%)	102	4,182	32.2%
Total	510	20,910	32.1%

exception is *Termination For Convenience*, which appears in 47.1% of test contracts against 33.1% of training contracts, a 14.0-point gap; the other outliers are all in the rare tail, where a shift of one or two contracts moves the percentage several points on its own. Since the split was drawn once at random and never re-drawn to look better, we report these gaps rather than tune them away — but they are worth carrying into Chapter 6, because a category over-represented in the test set carries more weight in the micro-averaged score than its training frequency would suggest.

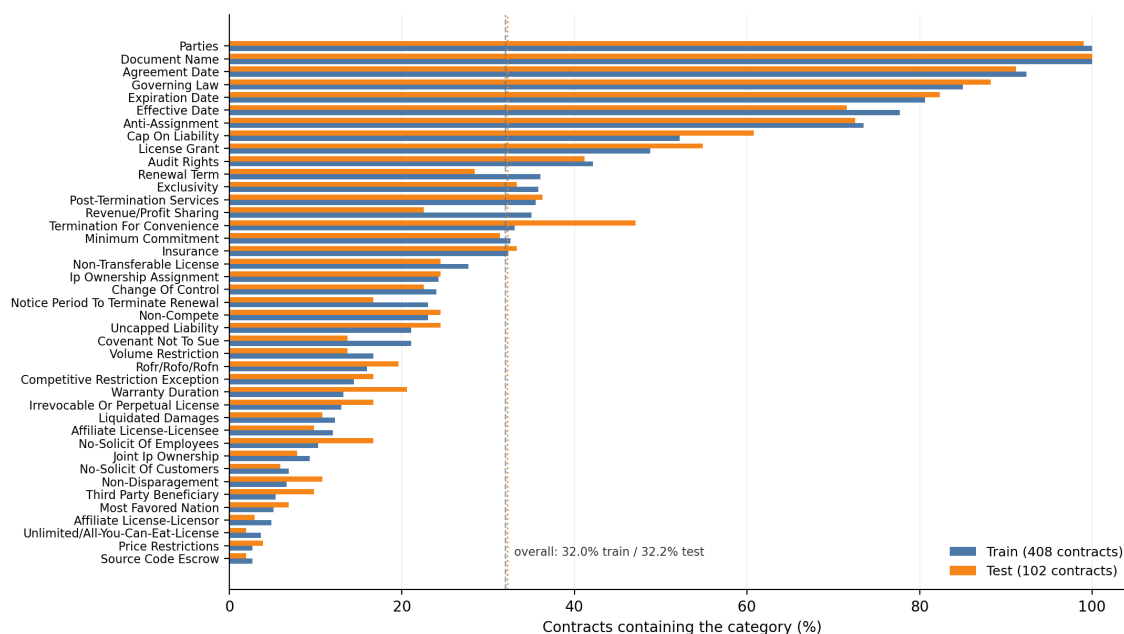


Figure 3.1: Per-category positive rate in the training and test halves of the contract-level split, sorted by training frequency. The two distributions overlap closely across all 41 categories, which supports the balance claim in Table 3.1 far more directly than the aggregate rates alone.

Chapter 4

Methodology

4.1 Pipeline Overview

The system is a staged pipeline (Figure 4.1). Contract text is windowed; a presence stage decides which of the 41 categories the contract contains; a span stage extracts the exact clause text for each detected category; a risk layer gives each clause a High/Medium/Low level; and a summarization stage rewrites the clause in plain English. The final output is a risk-prioritized report. Figure 4.1 names the stages and the models behind them; Figure 4.8 at the end of this chapter repeats the same chain with the concrete data object that crosses each boundary, which is the version to consult when reading the implementation.

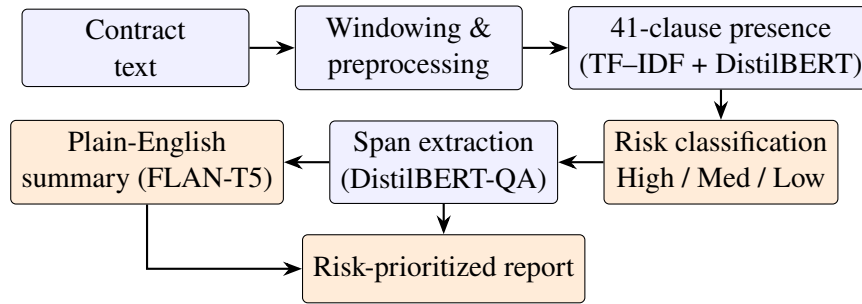


Figure 4.1: The end-to-end pipeline. Shaded (orange) stages — risk classification, plain-language summarization, and the prioritized report — are the layers this work adds beyond CUAD’s original clause-extraction task.

One principle runs through everything in this chapter: **build a strong, cheap baseline first, and measure feasibility ceilings before spending training time**. Both habits changed the design in concrete ways, as the next sections show.

4.2 Stage 2A: Presence Classification

4.2.1 A strong baseline first: full-document TF-IDF

Classical bag-of-words models have no input-length limit, so a TF-IDF representation gets to read the *entire* contract — an advantage no windowed transformer has. We fit a TF-IDF vectorizer (20,000 features, word uni- and bi-grams, sublinear TF, min_df = 2) on the 408 training contracts and train one class-balanced logistic regression per category. Legal

drafting is extremely formulaic (“*This Agreement shall be governed by the laws of the State of ...*”), so distinctive vocabulary alone carries a lot of signal. This baseline sets the bar every neural model has to beat.

4.2.2 Measuring the retrieval ceiling — and dropping retrieval

The obvious way to hand a long contract to a transformer is retrieval: slide 2,000-character windows with a 1,500-character stride over the text (the windowing scheme itself is drawn in Figure 4.3), score each window by TF-IDF cosine similarity to the category query, and give the model only the best window or windows. The query uses only information available at inference time — the category name or its CUAD question, never the answer — so there is no leakage.

Before training anything, we measured the *retrieval hit-rate*: how often does the selected window actually contain the gold clause? Figure 4.2 shows what the measurement is doing and why the answer is a hard bound. Whatever this number is, it is an upper bound on the downstream model’s recall, because a clause sitting in a window the retriever never selects cannot be found by any model placed after it. The best strategy we found (category name as query, top-3 windows) reached only **64.0%** (Table 4.1). Interestingly, the CUAD question text did worse than the bare category name, because the questions share so much boilerplate that all the category queries end up nearly identical. A recall ceiling of 0.64 sits below the baseline’s overall performance (≈ 0.78), so a retrieval-windowed transformer was going to lose before it was ever trained. The measurement took seconds and saved us a wasted training run.

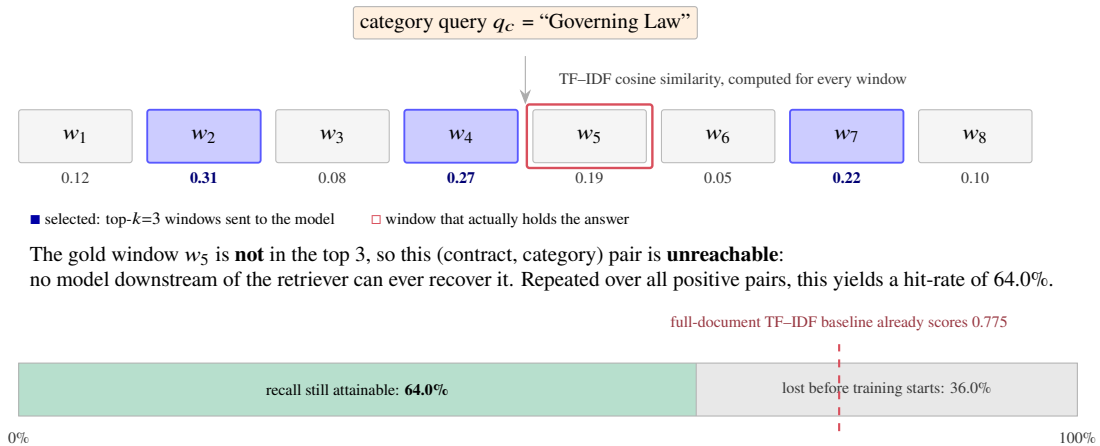


Figure 4.2: Why the retrieval ceiling settled the design. *Top*: the query scores every window and only the top- k go forward; whenever the gold-bearing window is not among them, the pair is lost no matter how good the downstream model is. *Bottom*: the resulting 64.0% ceiling sits *below* what the cheap full-document baseline already achieves (0.775, dashed line), so retrieval could not have won — which is why it was dropped before any training time was spent on it.

Table 4.1: Retrieval hit-rate on positive (contract, category) pairs — the recall ceiling a retrieval-based design would impose.

Query strategy	Top- k	Hit-rate
CUAD question text	1	38.6%
Category name	1	44.6%
Category name	3	64.0%
Name + question	3	61.5%

4.2.3 Sliding-window max-pooling (multiple-instance learning)

To get rid of the ceiling we trained a *window-level* classifier instead: given the pair (category question, window), predict whether that window contains a clause of the category. Figure 4.3 shows how the windows are generated. A 2,000-character window advancing in 1,500-character steps leaves a 500-character overlap between neighbours, and that overlap is what guarantees no clause can fall between two windows and disappear: the median annotated clause is 196 characters (Section 3.3), comfortably shorter than the overlap, so every clause lands wholly inside at least one window.

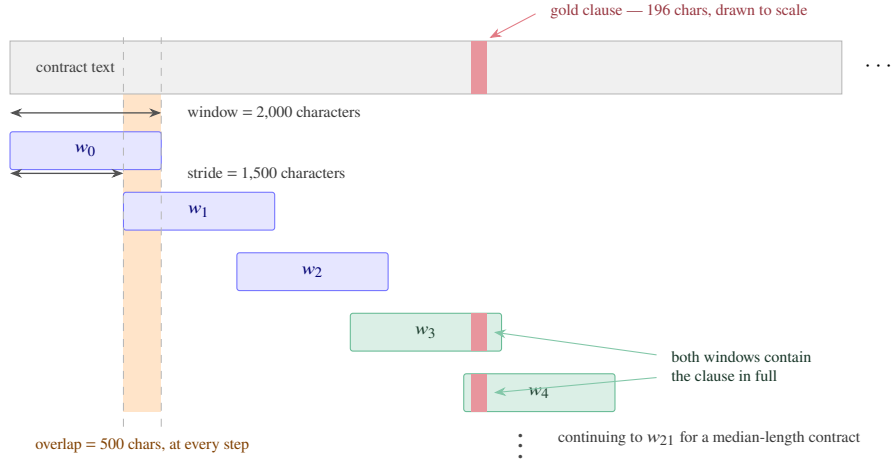


Figure 4.3: Sliding-window generation, drawn as a staircase so the geometry is explicit. Each 2,000-character window starts 1,500 characters after the previous one, so consecutive windows always share a 500-character overlap (orange column). Because that overlap is larger than the median annotated clause (196 characters), no clause can fall through the gap between two windows: here the clause lands inside *both* w_3 and w_4 in full. This is what removes the recall ceiling of Figure 4.2, and it is bought at the cost of scoring every window. A median-length contract produces 22 windows; the mean across the corpus is 35, and the longest contract yields 226.

Training labels come from the gold character offsets — any window containing an annotated span is positive, and for each (contract, category) bag we sample two negative windows. Using gold offsets this way is ordinary supervision, not leakage: the labels shape the training targets, but they never touch the model’s input, and nothing gold is used at inference. From

the 408 training contracts this gives 53,662 window examples (39.2% positive).

At inference the model scores *every* window of the contract, and the document-level prediction is simply the maximum window probability — the multiple-instance learning decision rule, drawn in Figure 4.4. The document is a *bag* of window instances; the bag is positive if any instance is positive; and max implements exactly that rule. Since the windows tile the whole document with overlap, every present clause is reachable, so the recall ceiling disappears by construction. The price is precision, and Figure 4.4 also shows why: the maximum is taken over every window, so a single spurious window anywhere in the document is enough to flip the whole contract to positive. With a median of 22 windows per contract and a mean of 35, the chance that some window crosses the 0.5 threshold by accident grows with document length — long contracts are structurally more likely to produce a false positive than short ones, independently of what they actually contain.

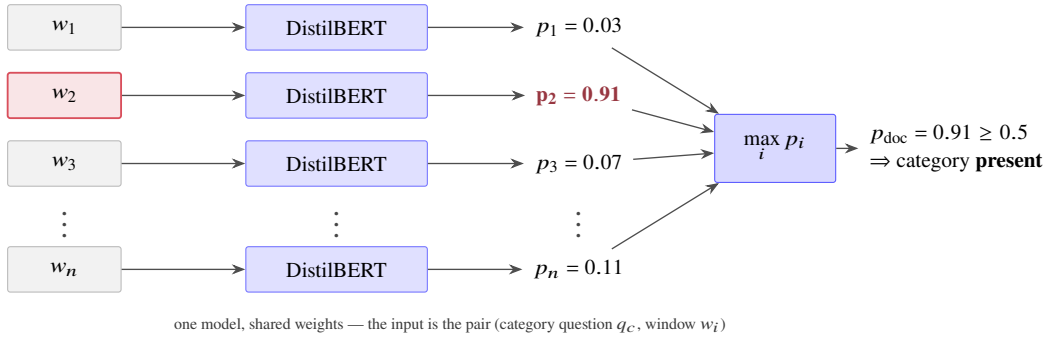
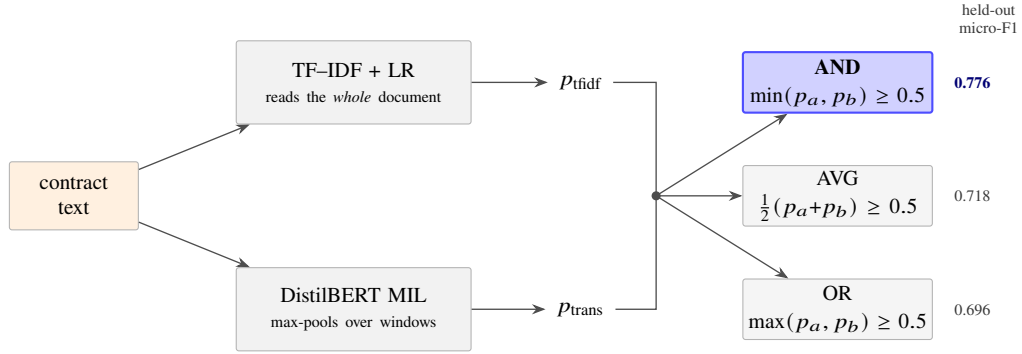


Figure 4.4: The multiple-instance learning decision rule. Every window of the contract is scored independently against the same category question by one shared DistilBERT, and the document-level probability is the maximum over windows. Here only the shaded window w_2 actually contains the clause, and its score alone determines the document-level answer. One firing window is enough to call the category present — which is what makes recall complete, and equally what makes precision fragile, since a single spurious window anywhere in a long contract produces a document-level false positive.

4.2.4 The conjunctive ensemble

When we compared the two models category by category, they turned out to be *complementary*: the transformer wins on semantically subtle categories (for example *No-Solicit of Employees* and *Liquidated Damages*), while the baseline wins on keyword-heavy ones. We combine them with **parameter-free** rules — AND (min of the two probabilities), OR (max), and AVG (mean) — chosen deliberately because they contain nothing tunable, so nothing can overfit to the test set. Figure 4.5 lays out the decision. The AND rule, which reports a clause only when both models agree, filters out each model’s uncorrelated false positives: for a spurious detection to survive, the lexical model and the windowed transformer must both make the same mistake on the same contract and category, and their mistakes are largely

independent. That recovers exactly the precision that max-pooling loses.



All three rules are **parameter-free**: there is nothing to fit, so the test set cannot tune them. AND fires only when *both* models cross the threshold, so a false positive has to be made twice, independently, to survive.

Figure 4.5: The conjunctive ensemble. The two presence models see the contract in fundamentally different ways — lexical evidence over the entire document versus semantic evidence over overlapping windows — and their errors are therefore largely uncorrelated. Requiring agreement (AND) discards the false positives that max-pooling introduces. The micro-F1 values on the right are the held-out results reported in Chapter 6, shown here only to indicate which rule the analysis selects.

4.3 Stage 2B: Span Extraction

Here the transformer has no substitute — a bag-of-words model cannot point at character positions. We fine-tune `DistilBertForQuestionAnswering`: the input is (CUAD question, window text) and the model predicts start and end token positions. Character offsets from `CUAD_v1.json` are converted to token labels through the tokenizer’s offset mapping.

Two different window sizes are in play in this thesis, and it is worth being explicit about the difference, since mixing them up makes the whole design look inconsistent. The 2,000-character window of Section 4.2.3 is a *presence* window: it tiles the document blindly, because at that point we do not know where anything is. The window used here is a *focus* window: 1,200 characters, and it is not tiled at all but re-centred on the gold answer, positioned so the answer begins roughly 150 characters in. Figure 4.6 draws the two on the same scale. The reason for the smaller, re-centred window is purely mechanical: it guarantees the target span survives truncation at the model’s 320-token limit, which a blind 2,000-character window does not — an answer sitting at the tail of a presence window would simply be cut off, and the model would be trained on an example whose answer is not present. Note also that this stage has no length ceiling at all: extraction happens *inside* a window that Stage 2A has already flagged as relevant.

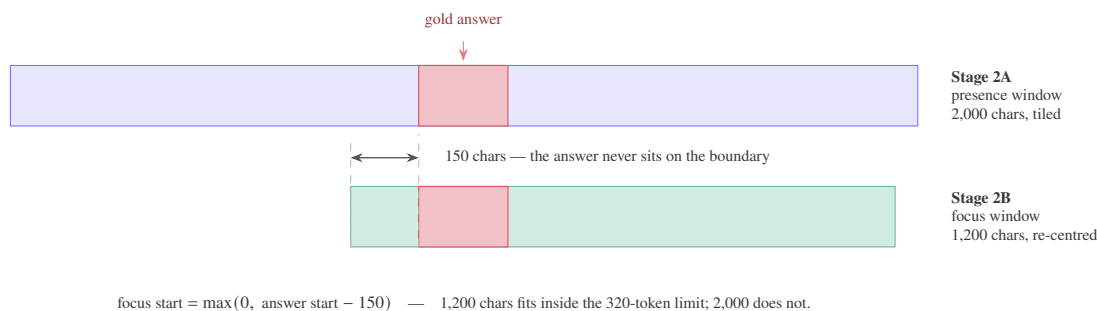


Figure 4.6: The two window sizes, drawn to the same scale. Stage 2A tiles the document with 2,000-character *presence* windows to find out *whether* a clause is there. Stage 2B then trains on a narrower 1,200-character *focus* window re-centred on the answer, so the span is guaranteed to survive tokenizer truncation. The two are not alternatives and never compete: the first locates a relevant region, the second reads inside it.

4.4 Stage 1: Plain-Language Summarization

CUAD gives us clauses but no plain-English rewrites to learn from, so we wrote our own seed: **41 gold templates**, one carefully phrased single-sentence summary per category, framed from the weaker party’s perspective. For *Anti-Assignment*, for instance, the template reads “You cannot transfer this contract to someone else without permission.” Every extracted training clause is paired with its category’s template, which yields 11,156 training pairs, and FLAN-T5-small is fine-tuned on the prompt “summarize in plain English: <clause>”.

We evaluate with ROUGE-L *and* with a side-by-side zero-shot-versus-fine-tuned comparison on the same clauses. The double evaluation is deliberate. With only 41 unique target sentences, a model can earn a high ROUGE score just by learning to classify the clause and emit its template word for word, without doing any real summarization. Chapter 6 shows that this is exactly what happens, and we report it as such.

4.5 Risk Taxonomy

Each of the 41 categories carries a fixed risk level — **High**, **Medium**, or **Low** — scored from the weaker party’s perspective, together with a one-line reason. The taxonomy (8 High, 22 Medium, 11 Low; Figure 4.7) is a transparent lookup rather than a learned score: it can be audited line by line, its reasoning is written down per category, and it cannot drift silently. The High categories fall into three buckets: money you could owe without limit (*Uncapped Liability*, *Liquidated Damages*, *Minimum Commitment*, *Most Favored Nation*); losing something you cannot get back (*IP Ownership Assignment*, *Irrevocable/Perpetual License*); and losing the freedom to earn later (*Non-Compete*, *Exclusivity*). The full clause-

by-clause table with reasons is in Appendix A.

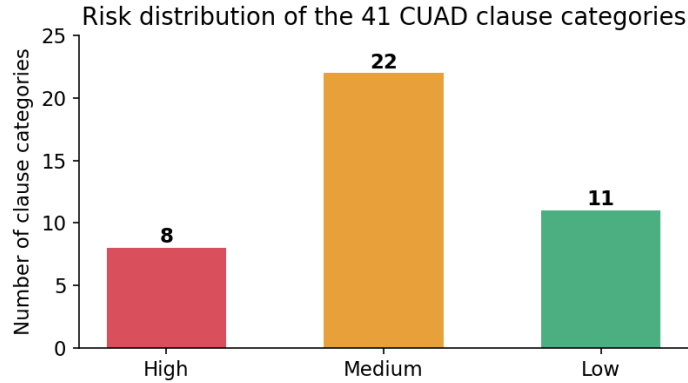


Figure 4.7: Distribution of the 41 CUAD clause categories across the three risk levels.

4.6 Leakage Discipline

With a dataset this small, leakage is the most dangerous way to fool ourselves, so we kept one explicit rule the whole way through: *labels may shape training targets, but they may never shape the input a model sees or any inference-time selection*. In practice this meant: the split is contract-level; retrieval queries use only the category or question; gold offsets label training windows but never build inputs or pick inference windows; and the ensemble rules are parameter-free, so the test set cannot tune them.

One consequence is visible in Figure 4.6 and deserves stating outright, because it is the one place where the rule looks like it might be broken. The focus window is positioned using the gold answer offset, which is a label. That is legitimate only because it happens exclusively at *training* time: it decides which text becomes a training example, never what the model reads at inference. At inference there is no gold offset, and Stage 2B simply runs over the presence window that Stage 2A selected. If re-centring were applied at inference the evaluation would be meaningless, since the model would be handed a window built around the answer it is supposed to find.

4.7 End-to-End Data Flow

Figure 4.1 names the stages; Figure 4.8 adds what actually moves between them. The shapes shown are for one contract of median length passing through the deployed system.

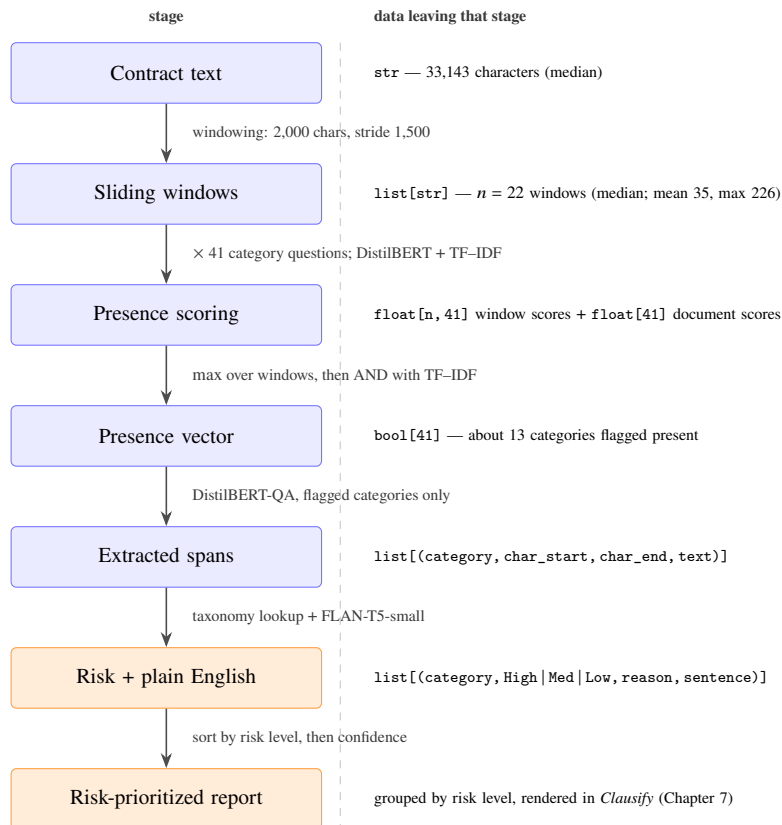


Figure 4.8: The pipeline of Figure 4.1 annotated with the object that crosses each stage boundary, for one median-length contract. The narrowing from `float[n, 41]` window scores to a `bool[41]` presence vector is where max-pooling and the AND rule act; everything downstream operates only on the roughly 13 categories that survive, which is what keeps inference fast enough for an interactive application.

Chapter 5

Model Training

All models were trained on the 408 training contracts (80%) only. Training ran on a single Apple-Silicon laptop using the MPS (Metal) backend with torch 2.8 and transformers 5.x; the summarizer ran on CPU for the memory reasons explained in Section 5.5. Table 5.1 collects the configurations.

Table 5.1: Training configurations for the three models.

	Presence (MIL)	Span (QA)	Summarizer
Backbone	DistilBERT-base	DistilBERT-base	FLAN-T5-small
Task head	Sequence classification	QA (start/end)	Seq2seq LM
Training examples	24,000 (of 53,662)	8,000 (of 11,156)	4,000 (of 11,156)
Max input length	256 tokens	320 tokens	256 in / 48 out
Epochs	2	2	2
Batch size	16	16	8
Learning rate	2×10^{-5}	3×10^{-5}	3×10^{-4}
Device	MPS	MPS	CPU
Wall-clock	$\approx 49\text{--}52$ min	≈ 23 min	≈ 8 min

5.1 Optimization Schedule

Table 5.1 gives the peak learning rates, but a peak rate on its own does not describe how a run behaved — the schedule that decays it does. All three models use the same optimizer configuration: AdamW with a **linear decay** schedule, **no warmup**, and gradient-norm clipping at 1.0. Figure 5.1 plots the three resulting schedules. The presence and span settings were read back from the `training_args.bin` saved beside each checkpoint rather than from our notes, so what is plotted is what actually ran.

Two choices are worth defending. Skipping warmup is normally risky, but these are short fine-tuning runs (1,000–3,000 steps) starting from pre-trained weights at small learning rates, where the usual justification for warmup — stabilizing early updates in a randomly initialized model — barely applies; the smooth loss curves in Figures 5.2 and 5.3 show no early instability. Clipping at a gradient norm of 1.0 was left on throughout, which matters more than it might seem on the MPS backend: it is the guard that keeps a single bad batch from producing the kind of divergence we hit with DeBERTa-v3 (Section 5.5).

We should be straightforward about one limitation here. A per-step gradient-norm trace

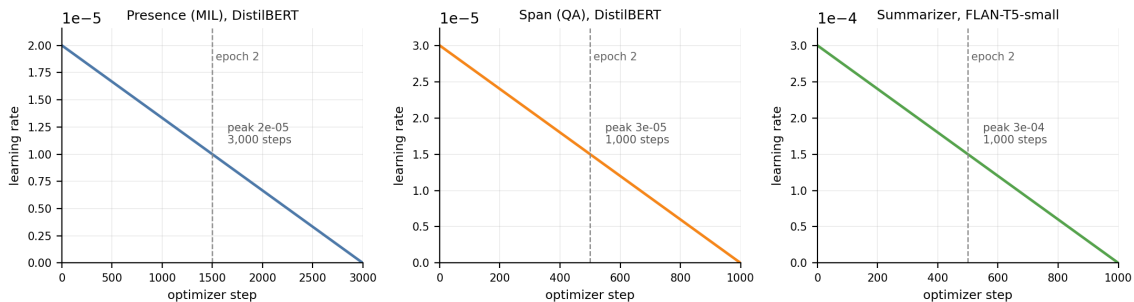


Figure 5.1: Learning-rate schedule for each of the three training runs: linear decay to zero with no warmup phase, over two epochs. The dashed line marks the start of the second epoch. Note the different vertical scales — the summarizer runs at 3×10^{-4} , an order of magnitude above the two DistilBERT runs, which is standard for T5-family models.

would be the natural companion to these curves, and it is the one diagnostic that would let a reader confirm the runs stayed numerically healthy rather than infer it from the loss. We did not retain it: the intermediate checkpoint directories were cleared after training, so only the final weights and the loss values quoted below survive. Re-running the three trainings with `logging_steps` capturing `grad_norm` would recover it in about ninety minutes of wall-clock time, and we note it as an omission rather than dress up what we have.

5.2 Presence Classifier

The TF-IDF + logistic-regression baseline trains in under a minute. On the 20% test set it reaches micro-F1 0.775, weighted-F1 0.777, macro-F1 0.572 over all 41 categories, and 0.666 over the 34 categories that have at least ten test positives. The gap between micro and macro comes from the near-few-shot tail of the dataset, not from a modelling failure.

The window-level DistilBERT was trained on a stratified subsample of 24,000 window examples (keeping the positive rate at 39.2%) so the run stays tractable on a laptop. Training and validation losses fell smoothly (0.320→0.261 train, 0.278→0.257 validation over two epochs; Figure 5.2), and the two curves stayed close together, so we saw no sign of overfitting at this scale.

5.3 Span Extractor

The QA model trained on 8,000 focused-window examples for two epochs. Before training we decode the token-labelled spans back into text for a few examples and check they match the gold answers — SQuAD-style pipelines make it very easy to get the character-to-token offset conversion silently wrong, and this small sanity check catches that.

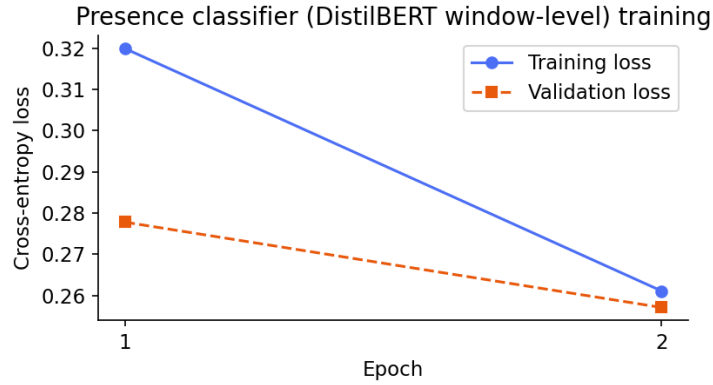


Figure 5.2: Training and validation loss of the window-level presence classifier.

5.4 Summarizer

FLAN-T5-small fine-tuned on 4,000 clause–template pairs in roughly eight minutes on CPU. The loss collapsed quickly (0.141→0.085 train, 0.096→0.075 validation; Figure 5.3). In hindsight this rapid collapse was the first hint of the template-memorization behaviour we analyse in Chapter 6 — with only 41 unique targets, the task is simply easy to fit.

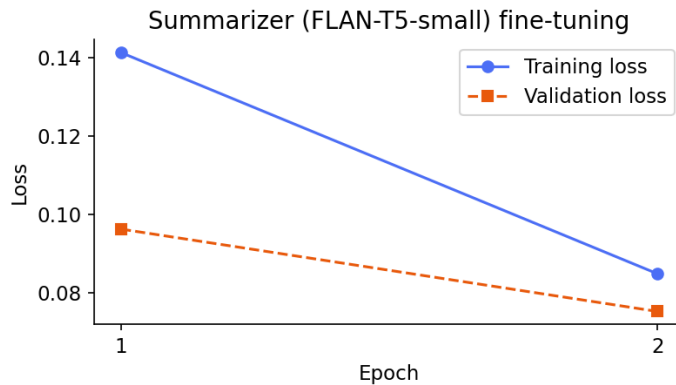


Figure 5.3: Fine-tuning loss of the FLAN-T5-small summarizer.

5.5 Engineering Notes: Training on Consumer Hardware

A few practical findings may save time for anyone reproducing this work on Apple-Silicon machines:

- **DeBERTa-v3 is unstable on MPS.** Our first-choice backbone produced NaN gradients on the MPS backend and was far too slow on CPU (about 89 seconds per step). We switched to DistilBERT; the method is unchanged, and on CUDA hardware the backbone string can simply be swapped back.

- **MPS memory does not release in-process.** After two transformer trainings in the same session, the summarizer hit the MPS memory ceiling even after we deleted objects and emptied the cache. The fixes that actually work are restarting the kernel or training the (small) summarizer on CPU, which is what we did.
- **API migration.** `transformers 5.x` renamed `Trainer(tokenizer=...)` to `processing_class=...`. We also disabled mixed precision (`fp16=bf16=False`) and cast loss logits to `fp32` for MPS dtype safety.
- **Reproducibility.** Every artifact — the split (`artifacts/split.json`), the fitted baseline (`artifacts/baseline.pkl`), and the three model checkpoints — is saved to disk, and a standalone evaluation notebook regenerates every number in Chapter 6 from those files alone.

Chapter 6

Testing and Results

Everything in this chapter is measured on the held-out 20% test set — the same 102 unseen contracts for every model. The numbers were regenerated by a standalone evaluation notebook that loads the trained models from disk, so no state left over from the training session can influence them.

6.1 Presence Classification

6.1.1 Model comparison

Table 6.1 and Figure 6.1 compare all the presence models on the 4,182 (contract, category) test decisions. Two things stand out. First, the windowed transformer *loses on its own* to the full-document TF-IDF baseline (0.694 vs. 0.775). Legal vocabulary is distinctive enough that lexical evidence over the whole document beats semantic evidence over windows, and max-pooling inflates false positives exactly the way Section 4.2.3 predicted. Second, the AND-ensemble — which reports a clause only when *both* models agree — filters out each model’s uncorrelated false positives and finishes on top (0.776). The margin over the baseline is small on 102 test contracts, so we do not lean on it: Section 6.1.2 quantifies exactly how small, and finds that this particular gap is indistinguishable from zero. The conclusion we stand behind is methodological: *simple lexical models are strong at document-level presence, and the transformer earns its place as a precision filter, not as a replacement.*

Table 6.1: Presence models on the 20% test set (micro-F1 over all 4,182 test cells).

Model	micro-F1
Ensemble AND (both must agree)	0.776
TF-IDF baseline (full document)	0.775
Ensemble AVG (mean probability)	0.718
Ensemble OR (either fires)	0.696
Transformer (max-pool) alone	0.694

6.1.2 How much of that ranking is real?

Table 6.1 is a leaderboard, and leaderboards invite a reader to treat the ordering itself as a result. On 102 test contracts a margin of 0.776 against 0.775 — roughly one decision in a

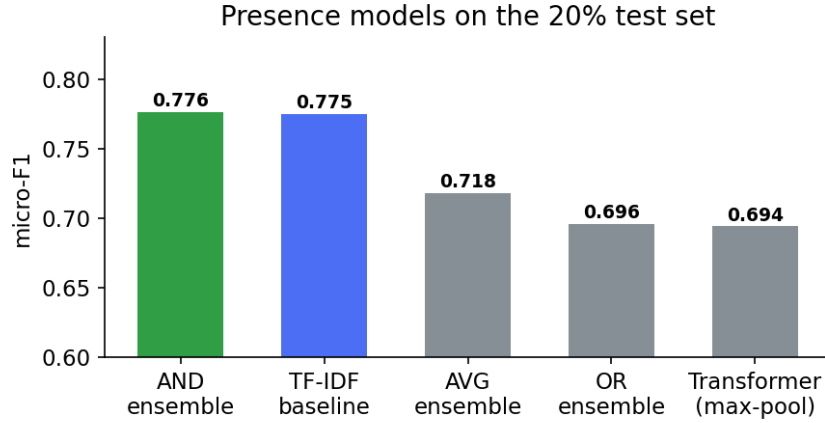


Figure 6.1: micro-F1 of all presence models on the held-out test set.

thousand — plainly cannot support that reading, so rather than leave the caveat as prose we measured it.

The natural tool is a **bootstrap confidence interval**, but the resampling unit matters. The 4,182 test decisions are not independent: 41 of them come from the same contract, and a contract that is unusual — unusually long, unusually drafted — moves all 41 together. Resampling individual (contract, category) cells would ignore that structure and produce intervals that are far too narrow. We therefore resample *contracts* with replacement (a cluster bootstrap), drawing 102 contracts from the 102 available, recomputing micro-F1 for both models on each resample, and repeating 10,000 times. Both models are scored on exactly the same resample every time, so the difference is *paired* — which is what allows a difference interval much tighter than the two individual intervals would suggest.

Table 6.2: Cluster bootstrap over the 102 test contracts (10,000 resamples). The paired difference is what settles each comparison; $P(> 0)$ is the fraction of resamples in which the first model wins.

Comparison	Difference in micro-F1	95% CI	$P(> 0)$
AND-ensemble vs. TF-IDF baseline	+0.0014	[−0.0068, +0.0089]	0.626
TF-IDF baseline vs. transformer	+0.0805	[+0.0626, +0.0992]	1.000
AND-ensemble vs. transformer	+0.0818	[+0.0654, +0.0987]	1.000

The three comparisons do not come out the same way, and the difference between them is the point.

The ensemble’s win over the baseline is not real. The interval is [−0.0068, +0.0089]: it straddles zero, and the ensemble is ahead in only 62.6% of resamples, barely better than a coin flip. Taken individually the two models score 0.776 (95% CI [0.759, 0.792]) and 0.775 (95% CI [0.758, 0.791]), intervals that almost entirely overlap. On this test set the AND-ensemble and the TF-IDF baseline are *statistically indistinguishable*, and we now state that

as a measurement rather than as a hedge. The top two rows of Table 6.1 should be read as a tie.

The transformer’s loss, by contrast, is real. The baseline beats the windowed transformer by 0.0805, with an interval of $[+0.0626, +0.0992]$ that is nowhere near zero, and it wins in every one of the 10,000 resamples. This matters for how the thesis’ two empirical claims should be weighted, because they are not on equal footing. The finding we stand behind most firmly — that a full-document lexical model beats a windowed DistilBERT at document-level presence classification — is the one that clears the significance test comfortably. The finding we were already cautious about — that the conjunctive ensemble edges past the baseline — is the one that does not. That our caution happened to point the right way is reassuring, but it was worth checking rather than assuming, and a reader should not have had to take it on trust.

None of this undercuts the case for the ensemble, because that case was never the 0.001 of micro-F1. The AND rule earns its place through the *error profile* it produces rather than its position on the leaderboard: it converts the transformer’s inflated false positives into balanced precision and recall, which is what the next section examines.

6.1.3 Confusion-matrix analysis

Our evaluation protocol called for a full confusion-matrix analysis of the final model. Table 6.3 gives the matrix for the AND-ensemble over all test decisions, Figure 6.2 shows the same matrix as a heatmap, and Table 6.4 lists every metric derived from it.

Table 6.3: Confusion matrix of the AND-ensemble on the 20% test set (4,182 decisions).

	Predicted Absent	Predicted Present
Actually Absent	TN = 2,523	FP = 311
Actually Present	FN = 296	TP = 1,052

Table 6.4: Test metrics derived from the confusion matrix.

Metric	Formula	Value
Accuracy	$(TP + TN)/\text{all}$	85.49%
Precision	$TP/(TP + FP)$	77.18%
Recall (Sensitivity)	$TP/(TP + FN)$	78.04%
Specificity	$TN/(TN + FP)$	89.03%
F1 score	$2PR/(P + R)$	0.7761

Precision and recall come out balanced (77.2% and 78.0%), which tells us the AND rule did its job: raw max-pooling gives up precision, and requiring agreement wins it back without

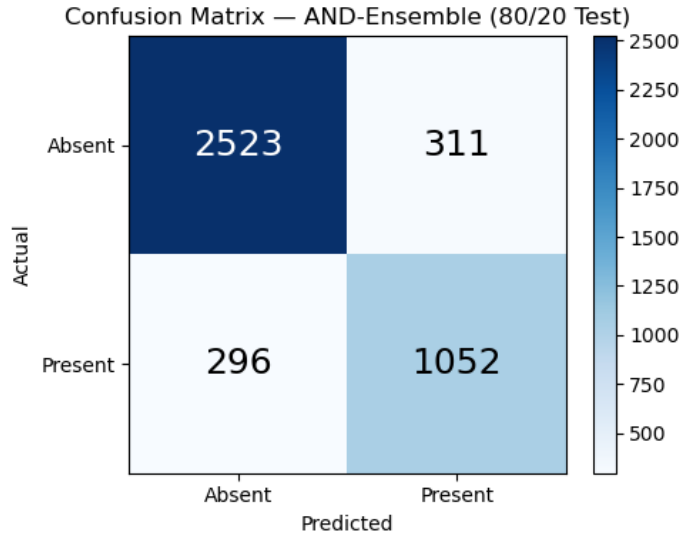


Figure 6.2: Confusion-matrix heatmap of the AND-ensemble on the 20% test set.

sacrificing recall. The specificity of 89.0% means the system rarely claims a clause that is not there — in a legal-review assistant, that is the more damaging kind of mistake.

6.1.4 Per-category behaviour

Figure 6.3 shows per-category F1 for the strongest categories. The frequent, formulaic ones are close to solved (*Document Name* 1.000, *Parties* 0.995, *Governing Law* 0.961, *Agreement Date* 0.958). At the other end, the five categories with at most seven test positives (*Most Favored Nation*, *No-Solicit of Customers*, *Price Restrictions*, *Source Code Escrow*, *Unlimited License*) all score 0.000. That number needs context: with two positive examples in the test set, a score of zero means the model missed two contracts, not that it is broken. This tail is a property of the dataset rather than of any architecture, and it is why we quote micro- rather than macro-averaged numbers in our headline results.

6.2 Span Extraction

We evaluated the QA model on all 2,667 answer-bearing test windows (Table 6.5). A token-F1 of 0.763, with 82.7% of predictions overlapping the gold span at $F1 \geq 0.5$, means that when the clause is in front of the model, it captures most of the right text roughly eight times out of ten. The lower exact-match score did not surprise us: annotated legal spans are long, and their exact boundaries are genuinely debatable (should a leading section number be included?), so content overlap is the metric that carries information here. For scale, the published CUAD baselines at this task use DeBERTa-v3-large, roughly five times the parameters of our distilled backbone.

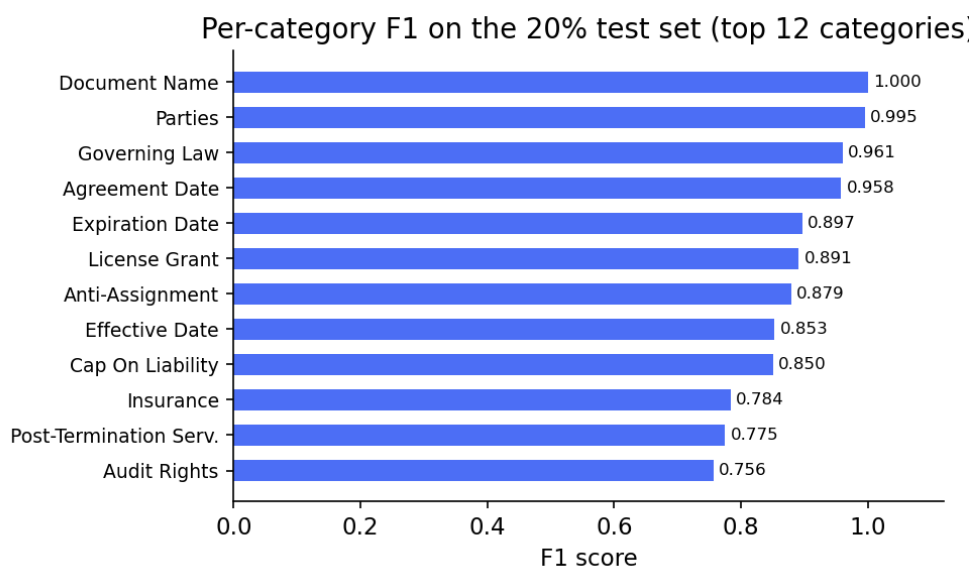


Figure 6.3: Per-category F1 on the test set for the twelve strongest categories.

Table 6.5: Span-extraction results on the 2,667 answer-bearing test windows.

Metric	Score
Token-level F1	0.763
Overlap (token-F1 ≥ 0.5)	82.7%
Exact Match	35.5%

Scope caveat. These numbers measure extraction quality *given an answer-bearing window*. The full-document benchmark in the CUAD paper (precision at 80% recall over all windows, including the answer-free ones) needs a heavier document-stride ranking evaluation that we did not run, so we make no claim on that metric.

6.3 Summarization

On 150 held-out test clauses the fine-tuned FLAN-T5-small scores **ROUGE-L 0.775**, well above the 0.40 target we set in the proposal. On its own, that number would overstate what the model actually learned. The qualitative comparison in Table 6.6 shows what is really going on: the fine-tuned model has learned to *classify* the clause and print its category template word for word. The output is clean, readable, and usually correct — but it is category-generic, and it ignores the details of the specific clause. Zero-shot FLAN-T5 does the opposite: it engages with the actual clause but only echoes the legalese back without simplifying it. With just 41 unique target sentences, fine-tuning quietly turned summarization into classification. The high ROUGE is real, but what it measures is template reproduction, and we would rather say so than let the metric overclaim. A genuinely abstractive legal summarizer

will need a larger, clause-diverse target set and a bigger model.

Table 6.6: Fine-tuned vs. zero-shot summarizer on the same test clause (abridged).

Input clause	“Any attempted assignment or delegation in violation of this section by either party without the prior written consent of the other will be void.”
Gold template	You cannot transfer this contract to someone else without permission.
Fine-tuned	You cannot transfer this contract to someone else without permission. (<i>exact template</i>)
Zero-shot	“a) Any attempted assignment or delegation in violation of this section ...will be void.” (<i>echoes the legalese</i>)

6.4 Summary Scorecard

Table 6.7 pulls together the held-out results of all three models. Every number was produced by the standalone test notebook running on the saved 80/20-trained models.

Table 6.7: Final held-out test results (408 training / 102 test contracts).

Stage	Model	Metric	Score
2A Presence	AND-ensemble (TF-IDF + DistilBERT)	micro-F1	0.776
2A Presence	AND-ensemble	Accuracy	85.49%
2A Presence	AND-ensemble	Precision / Recall	77.18% / 78.04%
2A Presence	AND-ensemble	Specificity	89.03%
2B Span	DistilBERT-QA	token-F1	0.763
2B Span	DistilBERT-QA	Exact Match	35.5%
1 Summarizer	FLAN-T5-small	ROUGE-L	0.775

6.5 Discussion: the Three Findings

1. For document-level *presence*, a simple lexical baseline beats a windowed transformer, and a paired cluster bootstrap puts that gap at +0.081 micro-F1 with a 95% interval of [+0.063, +0.099] — it is not a sampling artefact. The transformer pays for itself only as a precision filter inside a conjunctive ensemble, and even there the ensemble’s headline margin over the baseline is statistically a tie; what the ensemble genuinely buys is a balanced error profile, not a higher score.
2. For *span extraction*, the transformer has no substitute and performs well even in distilled form — this is where the neural model genuinely earns its computational cost.

3. For *summarization*, a tiny template seed quietly turns the task into classification. A high automatic metric should not be believed until the outputs have actually been read.

Chapter 7

Clausify: the Demonstration Application

7.1 Purpose and Design

The final deliverable is *Clausify*, a Streamlit web application that runs the two trained transformer models live on any contract the user provides and presents the findings as a risk-prioritized report — the pipeline of Figure 4.1, made usable. The design goal is simple: a non-lawyer should see the “watch out before signing” items first. Detected clauses are grouped **High** → **Medium** → **Low** and sorted by model confidence within each group.

7.2 User Workflow

1. **Add a contract.** A toggle offers two input modes — paste the contract text directly, or upload a .txt file (Figure 7.1). A confidence-threshold slider (default 0.50) controls how conservative detection should be.
2. **Analyze.** One click runs both models end-to-end; a progress bar tracks the presence scan across the 41 categories.
3. **Read the report.** A summary bar counts the detected clauses per risk level (Figure 7.2), and below it each detected clause appears as a card.

Each clause card shows four things: the category name with a colour-coded risk badge; the one-line plain-English reason that category carries its risk (from the taxonomy in Appendix A); the presence model’s confidence drawn as a bar; and the exact clause text the span model extracted, quoted from the user’s own contract (Figure 7.3). An expandable table lists all 41 categories with their risk level and raw presence score, so nothing the models computed is hidden from the user.

7.3 How the Models Run Inside the App

The application reproduces the inference path from training exactly:

1. **Windowing.** The contract is cut into 2,000-character windows with a 1,500-character stride — the same scheme used in training — capped at 30 windows so very long

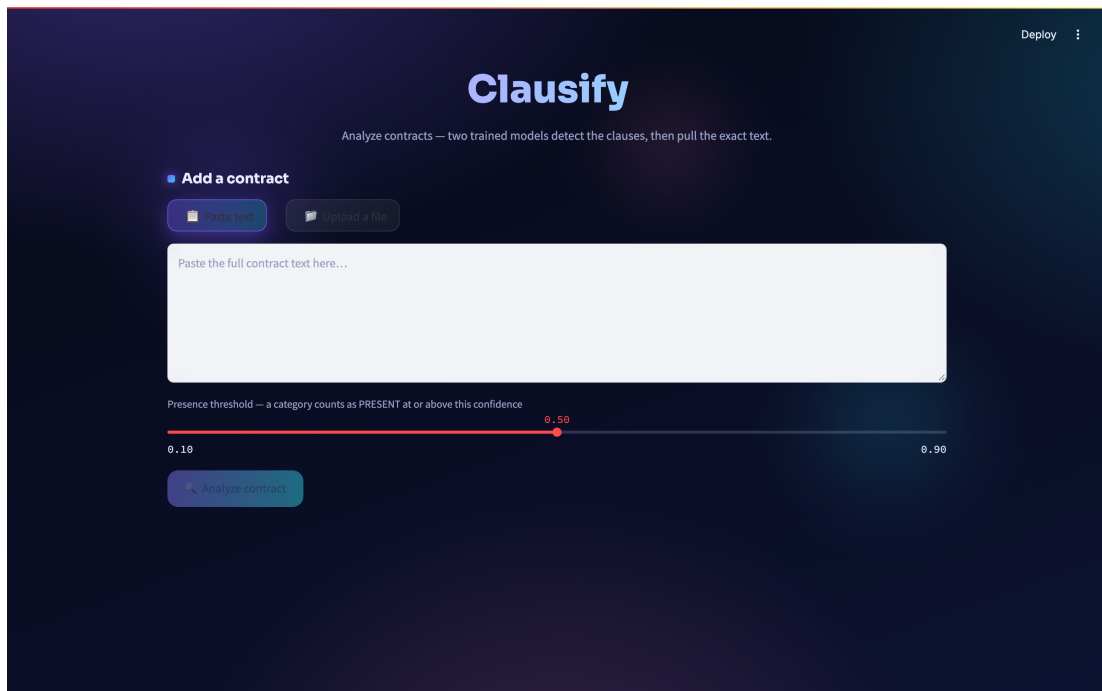


Figure 7.1: The Clausify input screen. The toggle selects between pasting contract text and uploading a file; the slider sets the presence-confidence threshold.

documents stay responsive.

2. **Model 1 — presence.** For each of the 41 categories, the exact CUAD question the model was trained on is paired with every window and scored by the window-level DistilBERT. The per-category document score is the max-pooled window probability; categories above the threshold are reported present, and the best-scoring window is remembered.
3. **Model 2 — span.** For each detected category, the DistilBERT-QA model runs over that best window (in 1,200-character focus chunks) and returns the highest-confidence span — the clause text shown on the card.
4. **Risk layer.** The detected category is looked up in the fixed 8/22/11 taxonomy for its level and reason.

All inference runs on CPU so the app works on any machine; a contract takes roughly 5–15 seconds once the models are loaded. The app loads the actual trained checkpoints saved by the training pipeline (`presence_mil/final` and `span/final`), and a single environment variable redirects it if the models are moved.

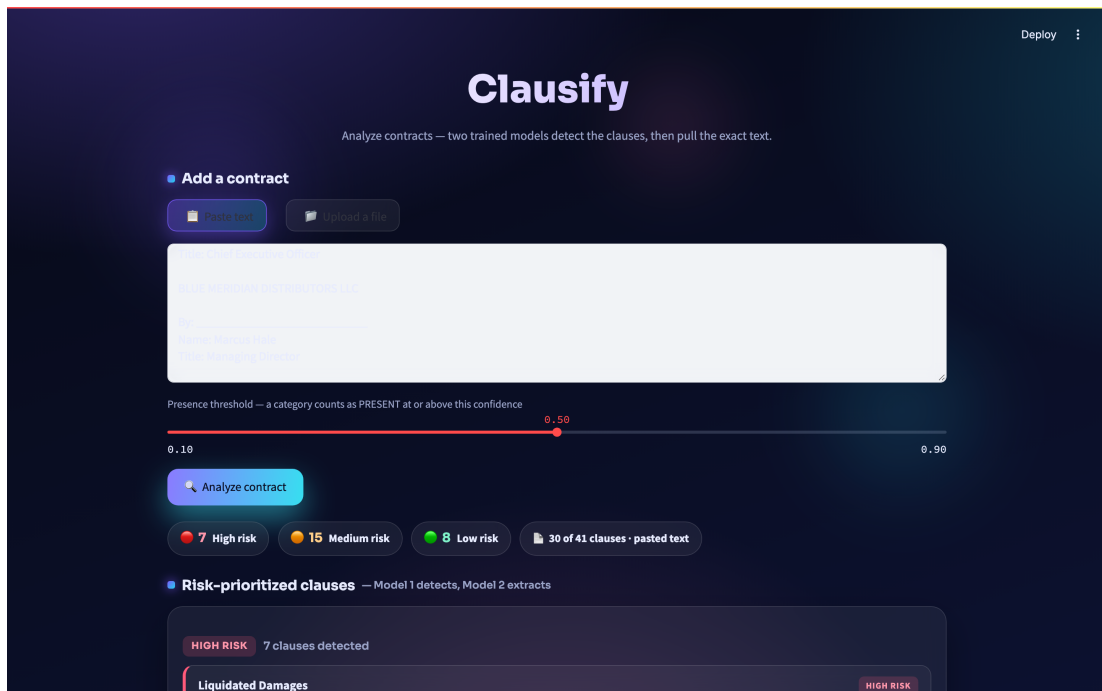


Figure 7.2: Results view on the synthetic test contract: the risk summary bar reports 7 High, 15 Medium, and 8 Low risk clauses (30 of the 41 categories detected), and the risk-prioritized report begins with the High-risk group.

7.4 Implementation Notes

The interface is a single-page Streamlit application (`app.py`) sitting on a small inference module (`model_utils.py`) that owns model loading, windowing, max-pooling, span extraction, and the risk table. Two test contracts ship with the app: a genuine CUAD contract, and a synthetic “Master Software License and Distribution Agreement” that we wrote specifically to exercise a wide spread of the 41 categories. On that synthetic contract the app detects clauses across all three risk bands and puts *Uncapped Liability*, *IP Ownership Assignment*, *Exclusivity*, and *Liquidated Damages* at the top of the report — exactly the before-you-sign warnings the design aims for.

7.5 Honest Limitations of the Demo

For interactive latency the app runs the transformer presence model alone rather than the AND-ensemble (the TF-IDF leg needs the fitted vectorizer, which the packaged demo leaves out). As Chapter 6 showed, the transformer alone over-detects slightly; the user can counter this by raising the threshold slider. Span boundaries inherit the distilled backbone’s approximate edges. And the risk levels are per-category, not per-instance: the taxonomy flags that a *Cap on Liability* clause deserves attention, but it does not read whether this particular cap is generous or punishing. All of this is stated in the app itself. What the demonstration does



Figure 7.3: High-risk clause cards. Each card combines the risk badge and reason, Model 1's presence confidence bar, and the exact clause text extracted by Model 2 from the user's contract — here *Uncapped Liability*, *Non-Compete*, *Minimum Commitment*, and *Irrevocable or Perpetual License*.

establish is that both models generalize to contracts they have never seen, and that the output can be presented in a form a layperson can act on.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

We set out to turn CUAD’s clause-extraction benchmark into something a contract signer can actually use: a system that finds the clauses, says how risky they are, and explains them in plain English. On a strict contract-level 80/20 split — 408 contracts for training, 102 never-seen contracts for testing — the pipeline we delivered reaches 85.5% accuracy (micro-F1 0.776, precision 77.2%, recall 78.0%, specificity 89.0%) for clause presence, token-F1 0.763 for exact clause extraction, and ROUGE-L 0.775 for plain-language summarization. Every one of those numbers can be regenerated from saved artifacts by a standalone evaluation notebook. The models run live inside Clausify, a web application that turns raw contract text into a risk-prioritized report in seconds.

Beyond the numbers, three lessons from this work seem likely to generalize. First, for document-level presence decisions over formulaic text, a full-document lexical baseline is a serious competitor: our windowed transformer lost to it outright by a margin a cluster bootstrap confirms is real, and an ensemble that uses the transformer as a precision filter only drew level with it rather than clearly beating it. Second, feasibility ceilings should be measured before models are trained — a retrieval hit-rate check that took under a minute (64%) killed an architecture that would otherwise have burned hours of training before failing. Third, automatic metrics need qualitative backup: our summarizer’s excellent ROUGE-L turned out to measure template reproduction, something we only saw by reading the outputs side by side, and we consider reporting that honestly worth more than the inflated headline.

8.2 Limitations

We downsized the backbone models (DistilBERT, FLAN-T5-small) to fit a laptop compute budget, so our absolute scores understate what the same method would achieve with larger backbones on CUDA hardware. The rare tail of CUAD (≤ 13 positive contracts for several categories) is effectively unlearnable at this dataset size, and our per-category results reflect that. The span evaluation measures extraction within answer-bearing windows, not the heavier full-document precision-at-recall benchmark. The summarizer reproduces templates rather than truly abstracting. And the risk taxonomy is per-category rather than

per-instance — it flags a category’s typical danger, not the severity of the specific wording in front of the user.

8.3 Future Work

- **Scale the backbones.** Re-run the unchanged pipeline with DeBERTa-v3-base/large and FLAN-T5-base on CUDA hardware to close the gap to the published CUAD baselines.
- **Full-document span benchmark.** Implement the document-stride ranked evaluation so we can report precision at 80% recall and compare with the CUAD paper apples-to-apples.
- **A genuinely abstractive summarizer.** Grow the 41-template seed into a clause-diverse target set (for example, reviewed LLM-drafted summaries) and re-fine-tune — the path from template classification to real simplification.
- **Cross-clause conflict detection.** Bring the rule engine designed in our pre-thesis phase (for example, *Exclusivity* vs. *Most Favored Nation*) into the deployed application.
- **Instance-level risk.** Move from per-category risk to per-clause severity by reading the extracted span itself — a cap’s amount, a non-compete’s duration.
- **Bangladeshi contracts.** Adapt the pipeline to local contract language and law, which is the deployment context that motivated this work in the first place.

Bibliography

- [1] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An expert-annotated NLP dataset for legal contract review,” in *Proc. 35th Conf. Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [2] The Atticus Project, “Contract Understanding Atticus Dataset (CUAD) v1,” 2021. [Online]. Available: <https://www.atticusprojectai.org/cuad>
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” in *5th Workshop on Energy Efficient Machine Learning and Cognitive Computing (NeurIPS)*, 2019.
- [6] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [7] H. W. Chung *et al.*, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.
- [8] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. EMNLP*, 2016, pp. 2383–2392.
- [9] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out (ACL Workshop)*, 2004, pp. 74–81.

Chapter A

Full Risk Taxonomy

The complete 41-category risk taxonomy (8 High, 22 Medium, 11 Low), scored from the weaker party's perspective, with the one-line reason shown to users in the Clausify application.

Table A.1: The 41 CUAD clause categories with risk level and reason.

Category	Risk	Reason
Non-Compete	High	Limits your future income and freedom to work.
Uncapped Liability	High	Unlimited damages — no ceiling on what you can owe.
IP Ownership Assignment	High	You lose ownership of a core asset for good.
Exclusivity	High	Locks you to a single source or buyer.
Most Favored Nation	High	You must always give the other side your best terms.
Liquidated Damages	High	Fixed penalties trigger automatically on breach.
Irrevocable or Perpetual License	High	A grant you can never take back.
Minimum Commitment	High	You are forced to buy or deliver a minimum amount.
Renewal Term	Medium	Auto-renewal can extend the contract unexpectedly.
Notice Period to Terminate Renewal	Medium	Miss the window and you are locked in for another term.
Competitive Restriction Exception	Medium	Carve-outs to competitive limits — read carefully.
No-Solicit of Customers	Medium	Restricts approaching the other side's customers.
No-Solicit of Employees	Medium	Restricts hiring the other side's staff.
Non-Disparagement	Medium	Limits what you can publicly say.
Termination for Convenience	Medium	The other side can walk away with notice.

Category	Risk	Reason
ROFR / ROFO / ROFN	Medium	First-refusal rights can constrain your options.
Change of Control	Medium	A sale or merger may trigger termination.
Anti-Assignment	Medium	You cannot transfer the contract without consent.
Revenue/Profit Sharing	Medium	You must share a slice of revenue or profit.
Price Restrictions	Medium	Limits what prices you may set or change.
Volume Restriction	Medium	Caps how much you can sell or distribute.
Joint IP Ownership	Medium	IP is co-owned — control is shared, not sole.
Non-Transferable License	Medium	The licence cannot be passed on.
Affiliate License (Licensor)	Medium	Licensor's affiliates are pulled into the grant.
Unlimited / All-You-Can-Eat License	Medium	Broad, unlimited licence scope — check who benefits.
Source Code Escrow	Medium	Source is held by a third party under conditions.
Post-Termination Services	Medium	Obligations continue after the contract ends.
Audit Rights	Medium	The other side can inspect your records.
Cap on Liability	Medium	Caps recovery — may under-compensate real losses.
Covenant Not to Sue	Medium	You waive the right to sue over listed matters.
Document Name	Low	Identifies the agreement — informational.
Parties	Low	Names who is signing — informational.
Agreement Date	Low	The signing date — informational.
Effective Date	Low	When terms start — informational.
Expiration Date	Low	When the contract ends — informational.
Governing Law	Low	Which jurisdiction's law applies — neutral.
License Grant	Low	You receive usage rights — generally favorable.
Warranty Duration	Low	States how long the warranty protects you.
Insurance	Low	Required coverage — a protection.
Affiliate License (Licensee)	Low	Extends rights to your affiliates — favorable.
Third Party Beneficiary	Low	Names an outside beneficiary — informational.

Chapter B

Repository and Artifact Layout

```
final project p3/                                -- research deliverable
|- PROJECT_REPORT.md / .pdf                      -- step-by-step project report
|- REPORT.md / .pdf                             -- findings & caveats report
|- METHODOLOGY.md / .pdf                       -- design-decision record
|- data/
|   |- train_80.csv                            -- 80% split (16,728 rows, 408 contracts)
|   |- test_20.csv                             -- 20% split (4,182 rows, 102 contracts)
|- notebooks/
|   |- train.ipynb                             -- training notebook (EDA -> 3 models)
|   |- test.ipynb                             -- STANDALONE evaluation notebook
|   |- eda.ipynb                              -- original combined notebook
|   |- build_artifacts.py                     -- retrains & saves all artifacts
|   |- artifacts/                             -- split.json, baseline.pkl
|   |- outputs/                               -- three trained model checkpoints
|- thesis/                                     -- this document (LaTeX)

final project app p3/                            -- Clausify demo application
|- app.py                                     -- Streamlit UI
|- model_utils.py                             -- inference + risk taxonomy
|- cat_questions.json                        -- 41 category->question strings
|- sample_contract.txt                      -- real CUAD test contract
|- test_contract.txt                       -- synthetic multi-clause contract
|- APP_REPORT.md / .pdf                     -- application report
```